



OPEN

An adaptive fall detection system based on ensemble learning using variants of YOLO V8 retinanet and DETR

S. Priya^{1✉} & K. Amshakala²

Object detection plays a critical role in automating visual recognition tasks like fall detection, which minimizes the need for constant human intervention. However, environmental challenges such as low-resolution images, crowded spaces, and varying image scales can decrease detection accuracy and increase the false positive rates. The proposed work aims to develop an adaptive, ensemble-based fall detection model named "YORD," which integrates modified versions of YOLO V8, RetinaNet, and baseline DETR. The goal is to improve detection accuracy and robustness across diverse environmental conditions. The YORD model leverages YOLO V8 for its speed and accuracy in real-time detection, DETR's attention mechanism for capturing falls from multiple angles and low-resolution images, and RetinaNet's focal loss strategy to address class imbalance and improve multi-scale detection. The dataset is enhanced using Albumentation to simulate real-world fall scenarios. The model is implemented in Google Colabs for streamlined experimentation and analysis. Experimental findings show that the YORD model is highly reliable and robust, with reduced false positives by utilizing the complementary strengths of each component model through ensemble learning. The YORD model achieved precise fall detection results across various challenging conditions, explaining the enhanced performance over the individual models. The YORD ensemble model effectively addresses the limitations of single-model fall detection systems, offering a robust solution for real-time applications. However, the system still faces some limitations in identifying the falls in the occluded images where the objects are only visible partially making it harder for the system to localize and classify them correctly. Future work will focus on fine tuning certain aspects of the model to focus on important parts of an image and give less importance to irrelevant or occluded areas.

Keywords Ensemble learning, Fall detection system, Object detection, YOLO V8, RetinaNet, DETR, Real-time monitoring, Transformer-based detection

Object detection is a relatively young subfield of computer vision that is crucial for a wide range of applications associated with different types of recognition techniques such as real time surveillance, self-driving cars and predictive analytics. Fall detection is one such technique in which the real time surveillance of people helps in precisely identifying the fall incidents and take immediate decisions without relying on the external assistance^{1,2}. Automated fall detection systems would enable the caregivers and medical responders to intervene at the first instance of a fall thus reducing the risks of other complications or even managing to save lives^{3,4}.

Despite the higher efficiency of the existing object detection models, single model predictions are susceptible to generate more false positives under challenging environmental conditions concerning resolution changes, multi scale variations^{5,6} and in denser regions. This kind of dynamic environmental changes generates high false positive rates and thus reduces the overall reliability of the system where the alerts must be accurate and fast^{7,8} in real time. In order to overcome the above challenges, the following multi-model approach leverages three object detection techniques namely YOLO V8, RetinaNet and DETR. The proposed adaptive ensemble fall detection approach aims at enhancing the detection performance across challenging environmental conditions^{9,10}, to make the fall detection process more reliable than compared to other standalone models^{11,12}.

Certain enhancements were made to the YOLO V8 and RetinaNet models. Each model brings unique strengths such as: Using YOLO V8 for real-time object detection with high accuracy^{13,14}, DETR for attention

¹Department of Information Technology, PSG College of Technology, Coimbatore, India. ²Department of Computer Science and Engineering, Coimbatore Institute of Technology, Coimbatore, India. ✉email: spriyait@yahoo.com

mechanisms that capture falls in different settings^{15,16} with low image resolutions, and RetinaNet's focal loss approach for balancing class backgrounds and multiscale variations. The ensemble approach utilizes these strengths, by taking the majority vote to counter the issue of high false positives and increase the overall accuracy across the difficult conditions^{17,18}. The detection accuracy and robustness is increased significantly as the ensemble models considers the best features from the individual models before making the final classification. Experimental results proved that the proposed technique was able to distinguish the varying fall activities^{19,20} precisely with lesser false positive rate and found to be more resilient across various environmental conditions.

The above Fig. 1 illustrates the work flow of the proposed method 'YORD'. The pre-processed images are provided as an input to the selected models i.e. modified YOLOV8, improved RetinaNet and baseline DETR. These three models will simultaneously begin to run and identifies the final output as fall/no-fall respectively. These individual model decisions are then provided as an input to the ensemble method which calculates the final outcome depending on the majority votes. In case the final decision is a 'fall' alerts are raised to the care takers.

The contributions of the study extend beyond traditional fall detection methodologies by introducing architectural optimizations, ensemble learning strategies, and real-time implementation enhancements that collectively improve accuracy, robustness, and practical applicability. Many existing fall detection approaches either rely on single deep learning models, handcrafted feature extraction, or sensor-based approaches, which struggle in complex environments such as low-resolution images, occlusions, multi-scale variations, and crowded spaces. In contrast, our work proposes a novel ensemble-based fall detection system, YORD, that integrates three advanced object detection architectures—YOLO V8, RetinaNet, and DETR—each optimized for specific challenges. The key contributions of this research include:

- *Development of an adaptive ensemble model:* Unlike prior fall detection methods that rely on a single deep learning model prone to false positives, we propose an adaptive ensemble system that leverages the complementary strengths of YOLO V8 (real-time efficiency), RetinaNet (handling class imbalance with Focal Loss), and DETR (global contextual understanding using transformers). This ensemble approach ensures higher accuracy, reduced false positives, and improved robustness in diverse scenarios.
- *Architectural optimizations in YOLO V8:* We enhance YOLO V8's feature extraction capabilities by incorporating Simplified Spatial Pyramid Pooling Fast (SimSPPF) and Bidirectional Feature Pyramid Networks (BiFPN), which allow better multi-scale fall detection. Unlike standard YOLO V8 models that struggle with detecting small-scale falls (i.e., individuals falling far from the camera), our modifications enable precise detection regardless of distance, resolution, or occlusions. Additionally, the removal of smaller detection heads optimizes computational efficiency, ensuring real-time applicability.
- *Enhanced RetinaNet with ResNet-D backbone:* Traditional RetinaNet suffers from performance degradation in low-resolution images, leading to misclassifications. We improve its robustness by replacing ResNet-50 with ResNet-D, which introduces anti-alias pooling operations to retain finer details in low-quality video frames. Furthermore, RetinaNet's Focal Loss ensures that the model prioritizes rare fall events, addressing the class imbalance problem commonly observed in real-world fall datasets.
- *Improved DETR with bipartite matching:* DETR-based object detection is often limited by redundant predictions in cluttered environments. To eliminate false positives, we integrate bipartite matching into DETR's prediction process, ensuring each detection is uniquely matched to ground-truth fall events, thereby reducing misclassifications in dense, multi-person settings. This refinement significantly outperforms traditional transformer-based approaches in complex surveillance environments.
- *Introduction of an optimized ensemble strategy:* Traditional ensemble methods in fall detection often rely on stacking or boosting techniques, which require additional meta-learners and increase computational overhead. Our majority voting mechanism is lightweight and computationally efficient, allowing all three models to run in parallel while ensuring real-time fall detection with an inference time of 0.39 s. This makes YORD more scalable and practical for real-time elderly monitoring, healthcare surveillance, and smart home applications.
- *Comprehensive performance analysis in challenging conditions:* Unlike prior works that evaluate fall detection models in controlled environments, our study tests YORD under adverse conditions such as low-resolution imagery, crowded spaces, multi-scale variations, and real-time streaming scenarios. Experimental results demonstrate that YORD achieves a higher mAP (84.2%) compared to single models (YOLO V8: 80.3%, RetinaNet: 80.4%, DETR: 80.3%), while significantly reducing false positives and false negatives.
- *Real-time alert system for practical deployment:* Our system is integrated with the Twilio cloud communication platform, enabling instant fall alerts to caregivers or emergency responders when a fall is detected. Unlike passive surveillance models, which only detect falls without an action mechanism, our approach ensures immediate intervention, making YORD highly applicable for real-world healthcare monitoring.

In this work section "Introduction" describes about the Introduction, Section "Related studies" describes about the related studies, Section "Dataset description" elaborates the input dataset, Section "Proposed methodology" depicts the proposed methodology, Section "Experimental result analysis" discloses the experimental result analysis, Section "Conclusions and future work" concludes this proposed work.

Related studies

Bin Li et.al²¹ has proposed a fall detection system based on the Convolutional Neural Networks and DETection Transformers. The author has extracted both the global and local features for the detection process. During the initial phase the local features are captured by the Convolutional Neural Networks and the Global features are captured by the Transformer models followed by the feature fusion stage which integrates both the local and

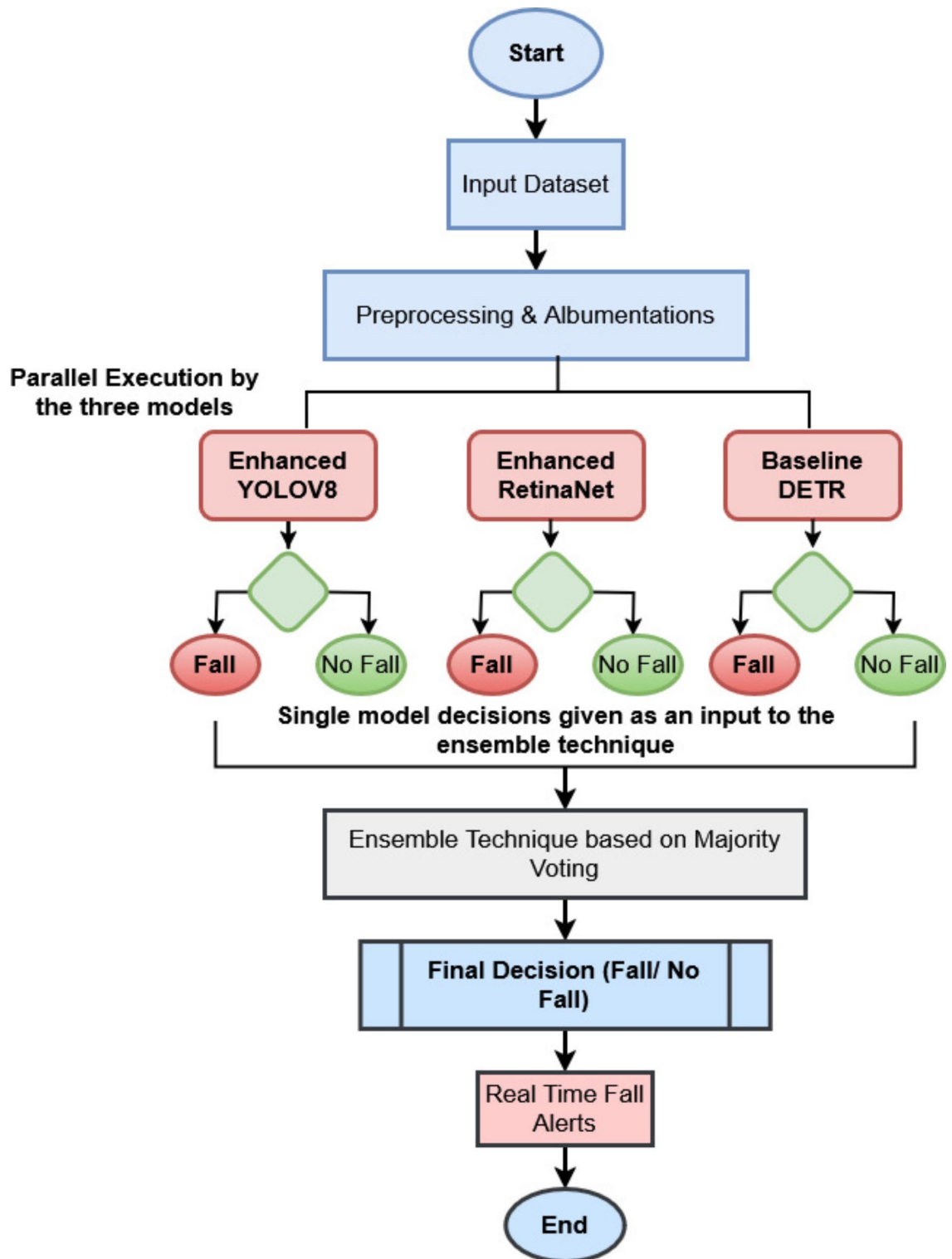


Fig. 1. Flow chart of the proposed 'YORD' model.

the global features. Metrics such as accuracy, precision and recall were considered for the evaluation. Input dataset was taken from the UP-Fall Detection and Le2i Fall Detection Datasets²¹ repositories respectively. The experimental platform was set up with Intel Xeon E5-2698 v4 processor, 256 GB of memory, and an NVIDIA Tesla V100 supported by the Ubuntu Operating System and deep learning libraries such as PyTorch 1.7.0. Experimental results have shown that the model yielded high fall detection rates of 99.55 and 99.75 percent individually for the chosen datasets. However, this model experiences some limitations in differentiating the actual falls with the lying down positions.

Xueyi Wang et al.²² has proposed a visual fall detection technique based on the egocentric approach through wearable cameras. The input dataset was collected manually from five volunteers. A camera was attached neck and the waists of each participant. Four types of fall activities and nine types of no fall activities were simulated by the participants under controlled environments. Total of 5,858 videos were collected from the first-person perspective. This input dataset was pre-processed by the pre trained convolutional neural networks followed by the temporal analysis and spatial feature fusion. Evaluation metrics such as accuracy, sensitivity and specificity were chosen for estimating the model's performance. The author was able to achieve an overall accuracy of 91.8% for sensitivity, 93.6% for specificity and 89.2% for classification²².

Yi Chu et al.²³ has proposed a fall detection approach based on WiFi channels. Traditional fall detection systems rely on the surveillance cameras or some wearable devices. These devices are prone to certain security vulnerabilities raising some potential privacy concerns. Wi-Fi Channel State Information (CSI) tools are used to collect the real time fall detection dataset. 700 CSI dataset samples were collected by performing daily life activities under four distinct environments. Dataset down sampling is chosen as the pre-processing technique. Convolutional Neural Networks were used to classify spectrograms generated from Wi-Fi CSI data. Experimental results proved that this model was able to achieve an average accuracy in the range of 96% to 99%. However, the proposed approach was able to perform well under controlled environments. The model generated some false positives in the real time fall detection process. While the system is non-intrusive, the use of Wi-Fi signals for monitoring may raise privacy concerns among users.

Anurag De et al.²⁴ proposed a novel fall detection approach based on spatial and temporal feature extraction process. Motion History images (MHI) highlighting the motion in the various parts of the image for processing the temporal features. Spatial Features were extracted by calculating the height to width ratio and the exact centroid for each and every person in the input image. Dual classification techniques such as threshold-based channel and key frame-based channels were used for the fall/no-fall classification process. The outputs of both the channels are fused together to make the final prediction. UR fall detection dataset was used for the implementation. Results proved that this two-channel classification model had an accuracy of 98.6% and a recall of 100% using UR Fall Detection Dataset²⁴. Though the model was able to perform well with the daily life activities it failed to identify the uncommon/rare fall positions.

Jiangjiao Li et al.²⁵ has introduced a novel fall detection approach based on the fusion of feature pyramid networks and transformer models. The proposed approach was able to effectively detect the features at multiple scales irrespective of the size and the position of the person in the image. Datasets from two benchmark repositories such as UR fall detection dataset and Le2i fall detection dataset was used for the experimental analysis. In the pre-processing phase normalization was performed over each and every input frame. This normalization was followed by the ReLU activation function. ReLU activation function helped to minimize the complexity of the model. In the next stage transformers were used to extract the spatial and temporal features of the input image. The extracted features were fused with the pyramid networks for achieving maximum efficiency. Experimental results have proved that the proposed model was able to achieve an average accuracy of around 99.61% and 99.33%. The experimental analysis was carried out using Intel Xeon E5-2698 v4 processor, 4 NVIDIA Tesla V100 GPUs, and the development language was python and pytorch deep learning library was used for the implementation. Though the model was able to perform well with multi scale images this system experienced some real time difficulties in distinguishing the actual falls and the normal lying down activities.

Narit Hnoohom et al.²⁶ has proposed a novel fall detection system based on the FDSNeXt model, built using deep residual networks. Participants wore inertial sensors and simulated the daily activities and some fall positions. Images from KFall repository with a total of 2729 daily motions and 2346 fall motions was also used for the experimental verification. After the pre -processing stage the FDSNeXt deep residual model was used for the classification purpose. The proposed method was validated against CNN, LSTM, and the combined CNN-LSTM models. The proposed approach achieved high accuracy rates of 91.87% for pre-impact fall detection and 92.52% for impact fall detection. Though the model was able to perform well under controlled environments, the FDSNeXt model had exhibited some computational constraints in the real time edge devices.

Thao V. Ha, et al.²⁷ has proposed a fall detection approach using a series of convolutional neural networks. The public dataset from UP Fall detection dataset was considered for the implementation. In the initial phase Mixture of Experts (MoE) was applied to the input images. The resultant image is passed to the series of Convolutional Neural Networks and max pooling layers followed by the ReLU activation function. Finally 3D CNN model was used to the input dataset for enhancing the spatio temporal features of the input images. ModNet model was also used for the foreground and background classification. The experimental platform was set up using Intel(R) Core (TM) i7- 7700 K 4 CPUs, running at 4.2 GHz with 16 GB of RAM and 48 GB of virtual memory, and Google Colab Pro environment. The input dataset was taken from UP fall detection dataset (294,678 samples) with six different kinds of daily life activities. Evaluation metrics such as True Positives, False Positives, Precision, Recall and F1 score was considered. A Weighted Average F1 score of 99.67% was obtained with account to the data imbalance concerns.

Jiangjiao Li et al.²⁸ has proposed a fall detection technique based on keypoint attentions. Object detection algorithms such as YOLOV3 and Open pose estimation were incorporated. The essential key points are extracted using YOLOV3 and Open pose estimation algorithms. The sliding windows were used to slice the extracted key points at the interval of 30 s. Finally, the results are strengthened by the adaptive key point attention mechanism. Average accuracy of around 99.73% was achieved for the UR Fall and Le2i datasets, respectively. Experimental platform was set up with a server with AMD Ryzen 9 5900HX processor, NVidia GeForce RTX3080 Laptop GPU, and 32 GB RAM in the Ubuntu OS and python language. Metrics such as accuracy, precision, recall, F1 score, specificity, sensitivity were considered for the evaluation purpose. Although the model gave better results the time taken for execution was very much longer than the conventional methods.

Shaobing Wang et al.²⁹ has proposed a dynamic fall detection approach using Patch-Transformer Network (PTN). These patch transformer networks work based on the wearable sensor algorithms. Local feature extraction is done using series of traditional Convolutional Neural Networks. Input dataset was taken from the SisFall and UnMib SHAR repositories. In the pre-processing phase daily life longer activities are split into shorter intervals for efficiency. The PTN network will fuse the Convolutional Network and Transformer layers for the detection process. Experimental platform was set up with the configuration Intel Core i5-12,400 CPU and NVIDIA GeForce GTX 3080 GPU. Metrics such as accuracy, precision, recall, F1 score, specificity, sensitivity were considered for the evaluation. A trapezoidal PTN algorithm based on the wearable sensor integrated using convolution layers with Transformer encoders obtained up to 99.86% accuracy on SisFall and UnMib SHAR datasets with low computational complexity.

Madhuri Agrawal et al.³⁰, has provided a real time fall detection system based on Longer Short Term Memory (LSTM) and YOLOV4. The input dataset was taken from two standard repositories such as UR Fall Detection Dataset and Multiple Camera Fall detection dataset. After the pre-processing phase the input features are extracted by the Convolutional Neural Networks. These features are then passed to the series of Bi directional LSTM's. Finally, softmax activation function was applied for the optimization. Metrics such as F score, specificity, Precision and Accuracy was considered. Average accuracy of around 96.9% was achieved using this method. Though the model excelled in identifying the suspicious fall activities, the model was not able to precisely identify the fall in the crowded environments.

Tingwei Chen et al.³¹ has proposed a fall detection technique based on Wi-Fi sensing. Signals generated from the WiFi transmitters were used for the fall detection purpose. In this work the input dataset was collected manually using ESP32 device. Daily life activities such as fall, sitting, standing, forward jumps and walk were considered. Linear interpolation was used as the pre-processing technique. The pattern changes in the input dataset was identified using the Convolutional Neural Networks and soft max activation function was used for the optimization. Experimental set up was done using Intel(R) Xeon(R) Silver 4210R CPU @ 2.40 GHz alongside a GeForce RTX 3090 GPU platform. Results have shown that the use of the Convolutional Neural Networks alongside reflected signals from the WiFi gave better results.

Haben Girmay Yhdego et al.³² has used the gait images to perform real time fall detection. Convolutional Neural Networks and Transformer models were used in the proposed approach. The limitations of the traditional CNN was overcome by the transfer learning approach. The proposed approach also addressed the challenges of the imbalanced datasets. The proposed approach also established a clear insight on the sensor placement techniques for higher efficiency. The model was evaluated using accuracy, precision, recall, and F1-score to assess their effectiveness in real-time fall detection scenarios. The model's result concludes that by combining the CNN with the transformer models helps to efficiently process the sensor data in real time. Though the model achieved higher precision the variability in the sensor placement has a vast impact on the accuracy.

Wafa Sulaiman Almukadi et al.³³ has proposed a computer vision based fall detection technique. The proposed approach combined a deep feature fusion with computer vision techniques (DFFCV-FDC) for optimization. Gaussian Filtering was used as a pre-processing technique. Hyper parameter tuning based on improved pelican optimization algorithm (IPOA) was incorporated after the fusion process. Final fall detection process was validated by an auto encoder. The model faced some difficulties in identifying the falls under the low lighting conditions.

Sha et al.³⁴ has proposed a novel light weight fall detection system based on deep learning. The traditional CNN networks were optimized by introducing Local Attention Module and Global Attention Module. The input dataset was collected from some benchmark repositories like COCO fall detection, UR fall detection dataset and some live video streams were also considered. The experimental platform was set up using single Nvidia A100 GPU and with Pytorch libraries. The proposed approach was light weight in nature with a detection accuracy of 95.1% and recall of around 93.3%. Although the proposed work seemed to be very much efficient it was not able to identify the falls in the small and in the occluded regions.

Jorge D. Cardenas et al.³⁵ multi class fall detection system based on doppler radars. This approach does not require any cameras or sensors to be carried out by the users. Instead Continuous Waves and radio frequencies emitted by the dopplers are used to monitor the unusual activities such as fall. These frequencies are processed by the multi class deep learning frameworks for the classification. Apart from the neural networks, conventional LSTM were also used for the classification purpose. Results proved that both the LSTM and CNN achieved an average accuracy of around 92.1%. Metrics such as accuracy, precision, recall and specificity were considered for the evaluation. The efficiency of the proposed approach depends upon the capability of the doppler devices.

From the analysis of the literature review it was identified that the models were not able to precisely identify the falls under varied environmental conditions such as low resolution, multi scale variations and crowded regions. The proposed work has introduced novel methods into existing object detection algorithms and the results have proved that the false positive rates are gradually reduced and the falls were able to be identified even in the above-mentioned adversarial circumstances. Summarization of the Literature Review is given in Table 1.

The integration of visual attention mechanisms into fall detection can improve the model's capacity to attend to relevant image areas and diminish the impact of background noise or occlusions. Attention or saliency models have been used in media processing for a variety of tasks, including quality assessment, fixation prediction, and multimodal saliency modeling, and hence may have potential applications in fall detection as well. Incorporating visual attention into YORD may be considered for future updates by adding attention mechanisms into DETR's transformer-based architecture, which already processes global image context effectively. Spatial and temporal attention layers may also be incorporated into YOLO V8 and RetinaNet to focus on prominent human motion features rather than background features. Another potential strategy is employing saliency-driven feature selection, whereby high-motion dynamics or body posture variation regions get prioritized attention to enhance detection performance in crowded settings. Whereas the present YORD model addresses ensemble learning for

Domain	Authors	Study	Dataset	Key findings
Global and Local Feature Extraction	Bin Li et al.,	Proposed a fall detection system combining CNNs and DETR. Local features were extracted by CNNs, while global features were captured by transformers. Feature fusion was used for enhanced detection	UP-Fall Detection and Le2i Fall Detection Datasets	Achieved high fall detection rates (99.55% and 99.75%). Struggled to differentiate falls from lying-down positions
Egocentric Wearable Cameras	Xueyi Wang et al.,	Developed a fall detection method using wearable cameras attached to participants' necks and waists. Four fall and nine non-fall activities were recorded	Manually collected (5,858 videos)	Achieved 91.8% sensitivity, 93.6% specificity, and 89.2% classification accuracy
WiFi CSI-Based Detection	Yi Chu et al.,	Used WiFi Channel State Information (CSI) for fall detection to address privacy concerns. CNNs classified spectrograms derived from CSI data	700 CSI samples across four environments	Achieved 96–99% accuracy. Produced false positives in real-time fall detection and raised privacy concerns
Motion and Shape-Based Features	Anurag De et al.,	Proposed a two-channel classification model using Motion History Images (MHI) for temporal features and height-width ratios for spatial features	UR Fall Detection Dataset	Achieved 98.6% accuracy and 100% recall. Struggled with rare/uncommon fall positions
Transformer-Based Spatiotemporal Features	Jiangjiao Li et al.,	Combined feature pyramid networks and transformers for multi-scale fall detection	UR and Le2i Fall Detection Datasets	Achieved ~ 99.61% and 99.33% accuracy. Struggled to distinguish falls from normal lying-down activities
Pre-Impact Fall Detection	Narit Hnoohom et al.,	Developed the FDSNeXt model using deep residual networks for pre-impact fall detection	KFall Dataset (2,729 daily motions, 2,346 falls)	Achieved 91.87% accuracy for pre-impact and 92.52% for impact fall detection. Faced computational constraints on edge devices
Mixture of Experts (MoE) with CNN3D	Thao V. Ha et al.,	Applied Mixture of Experts (MoE) with CNN and ModNet for fall detection	UP Fall Detection Dataset (294,678 samples)	Achieved a Weighted Average F1 score of 99.67%. Addressed data imbalance concerns
Keypoint Attention and Temporal Analysis	Jiangjiao Li et al.,	Used YOLOV3, OpenPose for keypoint detection, and adaptive attention mechanisms for improved fall detection	UR and Le2i Fall Detection Datasets	Achieved ~ 99.73% accuracy but had longer execution times
Patch-Transformer Network (PTN)	Shaobing Wang et al.,	Proposed PTN combining CNNs and transformers for fall detection using wearable sensors	SisFall and UnMib SHAR Datasets	Achieved 99.86% accuracy with low computational complexity
Visual Attention-Guided Bi-Directional LSTM	Madhuri Agrawal et al.,	Developed a system using LSTMs and YOLOV4	UR and Multiple Camera Fall Detection Datasets	Achieved 96.9% accuracy. Struggled to detect falls in crowded environments
WiFi Sensing for Fall Detection	Tingwei Chen et al.,	Leveraged WiFi sensing with CNNs for non-intrusive fall detection	Manually collected using ESP32 device	Demonstrated improved detection accuracy using WiFi signal patterns but raised privacy concerns
Wearable Sensors for Fall Detection	Haben Girmay Yhdego et al.,	Fall detection using sensor-based algorithms (details to be completed)	Details to be added	Achievements to be completed

Table 1. Summarization of the literature review.

improving robustness, subsequent work could combine attention-based optimizations to eliminate further false positives and enhance fall localization under dense and low-resolution settings.

Blind image quality estimation via distortion aggravation

Xionghuo Min et al.³⁶ has proposed the Blind Image Quality Estimation via Distortion Aggravation for efficient image processing. This study introduces multiple pseudo-reference images by further degrading the distorted image in various ways and degrees. The method evaluates image quality by comparing the distorted image to these pseudo-references. This method introduces additional distortions to an already distorted image to create multiple pseudo-reference images. The quality of the original distorted image is then assessed by comparing it to these aggravated versions. Performance is evaluated using standard metrics such as the Spearman Rank-Order Correlation Coefficient (SROCC) and the Pearson Linear Correlation Coefficient (PLCC), which measure the correlation between the predicted quality scores and human subjective ratings. The quality of the pseudo-reference image heavily depends on the degradation process used, which may not accurately represent all types of distortions.

From the analysis of the literature review it was identified that the models were not able to precisely identify the falls under varied environmental conditions such as low resolution, multi scale variations and crowded regions. The proposed work has introduced novel methods into existing object detection algorithms and the results have proved that the false positive rates are gradually reduced and the falls were able to be identified even in the above-mentioned adversarial circumstances.

Dataset description

The input dataset was taken from three distinct sources. The first source was taken from <https://universe.roboflow.com/roboflow-universe-projects/fall-detection-ca3o837>. The dataset comprises 4,497 images from Roboflow's fall detection dataset, which consists of a wide range of fall situations. The images cover different types of falls like forward falls, backward falls, side falls, and slips, recorded under various environmental conditions. The dataset also comprises images with different resolutions, lighting conditions, and camera angles to provide a real-world representation of actual fall accidents. Also, it includes single-person and multi-person fall situations, which enable the model to generalize more in crowded scenes where fall detection is generally more difficult.

The second set of images was taken from <https://www.kaggle.com/datasets/uttejkumarkandagatla/fall-detection-dataset38>. This dataset included a range of class labels such as fall, sitting, standing, walking, and fall

detected. The images in the dataset were captured in multiple perspectives to increase the robustness of the input dataset. The third dataset was taken from <https://falldataset.com>³⁹. This dataset consisted of a total of 22,636 images without any repetitions but few images are flipped horizontally to improve the en-richness of the input dataset. The images in the dataset were recorded in five different rooms with 5 different angles. Five participants were involved in the recording of this dataset. Figure 2 illustrates the sample dataset instances.

Albumentation

Albumentation helps to improve the versatility of the dataset with the help of several augmentation techniques like flipping, horizontal and vertical flips and rotations, random scale, brightness, contrast, rotation and blur. This allows the model to adapt more effectively to diverse scenarios and variations in the input data including changes in angles, lighting conditions and low-resolution images. This technique helps to enhance the model's accuracy and recall across diverse environmental conditions.

Proposed methodology

The proposed methodology, termed “YORD” (YOLO-RETINANET-DETR), is a novel ensemble-based fall detection approach which leverages the advantages of three distinct object detection models namely enhanced YOLO V8, enhanced RetinaNet, and Baseline DETR. Unlike the other fall detection models, the proposed approach emphasis on the ensemble technique, which combines multiple state of the art models before making a final decision. Considering the results of more than one model significantly reduces the false positive rates and henceforth the detection robustness is increased which is very crucial for the real time fall detection. YOLOV8 is renowned for its greater speed and higher accuracy there by making it as a suitable algorithm for real time fall detection systems requiring immediate response. Though YOLOV8 excels in real time fall detection it has few setbacks in identifying the falls across the multi scale variations generated by wide angle surveillance footages. Existing YOLOV8 relies on the grid-based detection which poses some restrictions in detecting the falls across the smaller images. As a result, the persons falling far away from the camera are missed. The proposed YOLOV8 helps in addressing this challenge by introducing Simplified Spatial Pyramid Pooling Fast (SimSPPF) across the YOLOV8's backbone. Further the incorporation of Bidirectional Feature Pyramid Networks (BiFPN) in the YOLOV8's neck region encourages the flow of the bidirectional information across the features to capture the falls at distinct scales. This is more significant to capture the multi scale falls where the person might fall closer to the camera and appears smaller in size or fall far away from the camera and appear to be larger in size. Though the proposed YOLOV8 effectively handled the multi scale variations it had some difficulties in detecting the falls



Fig. 2. Sample dataset instances.

in the lower resolution images and in crowded images. To handle this challenge the proposed model introduces enhanced RetinaNet with ResNet D backbone. In the low-quality video frames ResNet D's enhanced residual connections helps to retain the finer details. This helps in precisely detecting the fall even in the low-resolution images. Fall detection systems often encounter real time class imbalance problems where the number of fall incidents is relatively lesser than the number of no fall incidents. RetinaNet's Focal loss feature helps in effectively handling the class imbalance issues with the help of the focal loss function.

Baseline DETection TRansformers (DETR) is used as a third model during the ensemble process. This transformer-based architecture makes use of the attention mechanism which helps in better identification of the fall positions even in the cluttered regions. DETR captures the global context in the images which helps in capturing the falls effectively across adverse environmental conditions. As a whole the proposed ensemble technique mitigates the single model errors and paves way for highly precise fall detection systems in real time.

The proposed YORD methodology starts with the data pre-processing step where the authors use Albumentation-based augmentations to model realistic variations. Consequently, each frame or image input to YOLO V8, RetinaNet, and DETR undergoes its decision-making on the possibility of falls independently. In general, the combination of detectors in YOLO V8 targets the objects more accurately and quickly; RetinaNet targets all objects while applying focal loss to reduce overemphasis on less frequent events; DETR targets and focuses on specific detected regions by attending to relevant in crowded scenes. These predictions were quantified using a majority voting system in which the final detection was based on the model with the most frequent prediction. This makes the decision ensemble highly accurate as well as reducing the number of false positives; which proves the solution is ideal for real-time fall detection.

Figure 3 demonstrates the suggested approach. In the first step, input images are preprocessed using the albumentation and augmentation method. The preprocessed images are provided as an input to the improved YOLOV8, improved Retina Net and baseline DETR model simultaneously. YOLOV8 is improved using SimSPPF and BiFPN incorporation. RetinaNet was tuned using the Resnet D module. These three algorithms will classify independently on their own classification result as fall/no fall. Single models' final classification results are paired together and provided as an input to the ensemble method. The ensemble method will output the final result using the majority votes provided by the three single models. Once the final decision is taken as a Fall proper real time alerts are sent to the respective care takers via the Twilio platform.

YOLOV8

Though YOLOV8 is considered as one of the most robust object detection models it exhibits certain limitations in terms of computational efficiency and feature extraction. Primary YOLOV8 predominantly relies on the Path Aggregation Networks (PAN), which is less effective in integrating multi scale contextual information and as a result the objects exhibiting significant scale variations goes unnoticed. As the prevailing YOLOV8 demands higher computational resources, running it over the low powered edge devices may not detect the falls on time leading to slower emergency response in real time.

To overcome these setbacks the proposed approach introduces Simplified Spatial Pyramid Pooling Fast (SimSPPF) into the backbone of the YOLOV8 network as shown in the Fig. 4.

The traditional SPPF utilizes pooling operations with distinct kernel sizes leading to increased computational complexity as follows,

$$O(SPP) = C * H * W * \sum_{i=1}^n k_i^2 \quad (1)$$

where C represents the input features, H and W represents the height and width of the distinct image resolutions, K represents the small, medium or large falls and n represents the pooling layers. From the Eq. (1) it is obviously observed that the complexity of the SPP computation is increased proportionally with the kernel size (i.e. small, large or medium size images) leading to increased response time.

In order to optimize this complexity for real time edge devices the proposed SimSPPF in the backbone layer reduces the complexity as shown in the Eq. (2)

$$O(\text{SimSPPF}) = C * H * W * k^2 \quad (2)$$

SimSPPF replaces the traditional multiple pooling operations with a single 5 * 5 max pooling operation thereby reducing the number of the redundant computations to make the detections much more faster and simpler in real time. Further to handle the multi scale variations with lesser complexity and higher efficiency the proposed SimSPPF makes use of the stride based down sampling technique where the spatial resolutions of the image is reduced at each stage as shown in the Eq. (3).

$$O(\text{SimSPPF}) = C * \sum_{i=1}^n \frac{H}{s_i} * \frac{W}{s_i} * k_i^2 \quad (3)$$

where s_i represents the stride for each pooling layer. This stride based pooling layers helps to ensure the smaller, larger and medium falls are captured without any miss. Thus YOLOv8's enhanced backbone enables extensive feature extraction, improving detection accuracy under challenging environments with distinct camera angles. Additionally, the smaller detection heads are also removed in the backbone region. This removal further lowered the computational complexities leading to lesser inference time. The removal of the smaller detection heads in

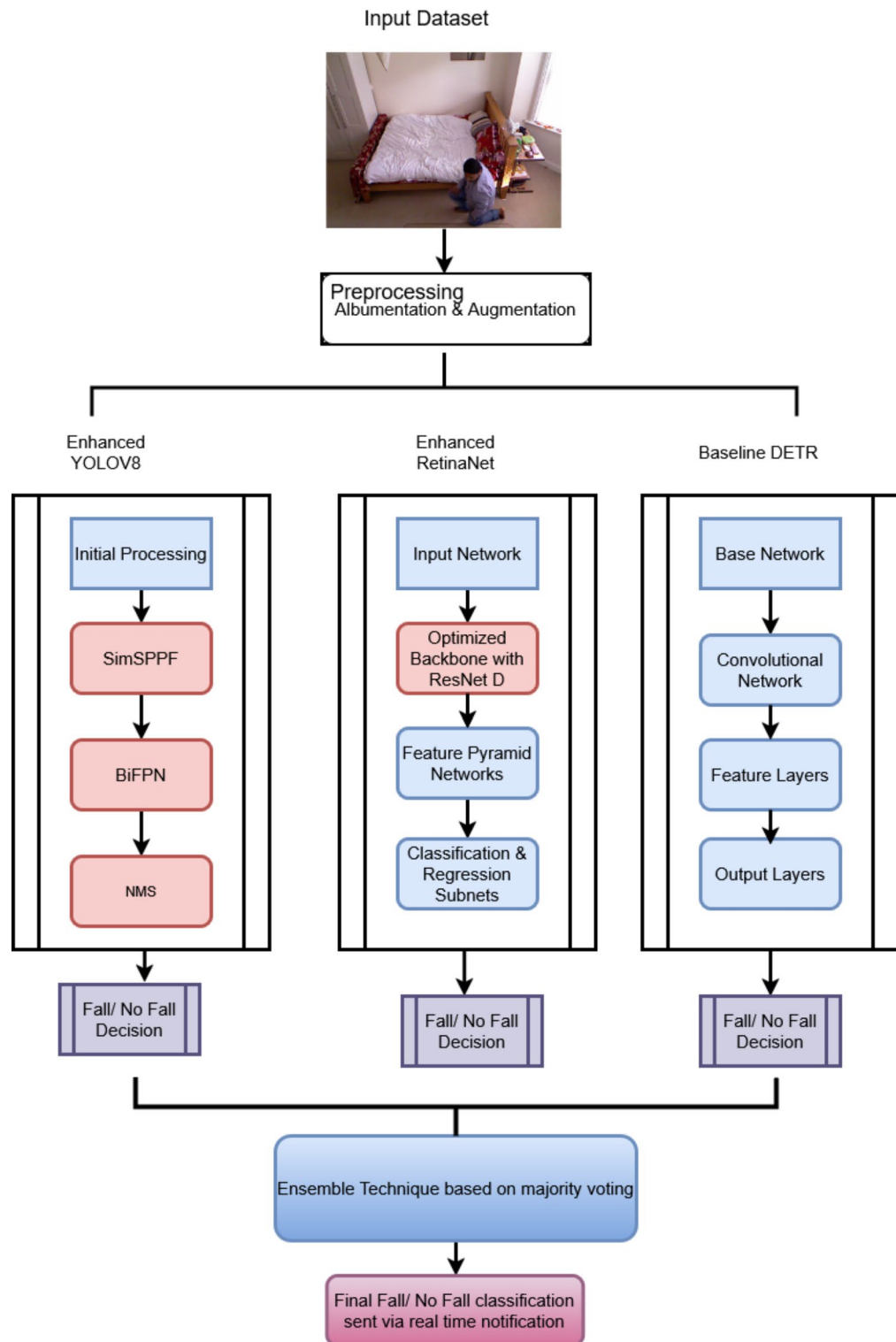


Fig. 3. Proposed methodology.

the YOLOV8's backbone does not have any impact on the accuracy as the BiFPN in the neck region scans the entire image in both the top down and bottom up directions for extensive object detection at various scales.

Advanced pooling operations for resilience enhancement

Let $F_1, F_2, F_3, \dots, F_N$ represents the extracted feature maps representing the diverse spatial scales as follows.

$$F_i 1^n = \text{Backbone}(I) \quad (4)$$

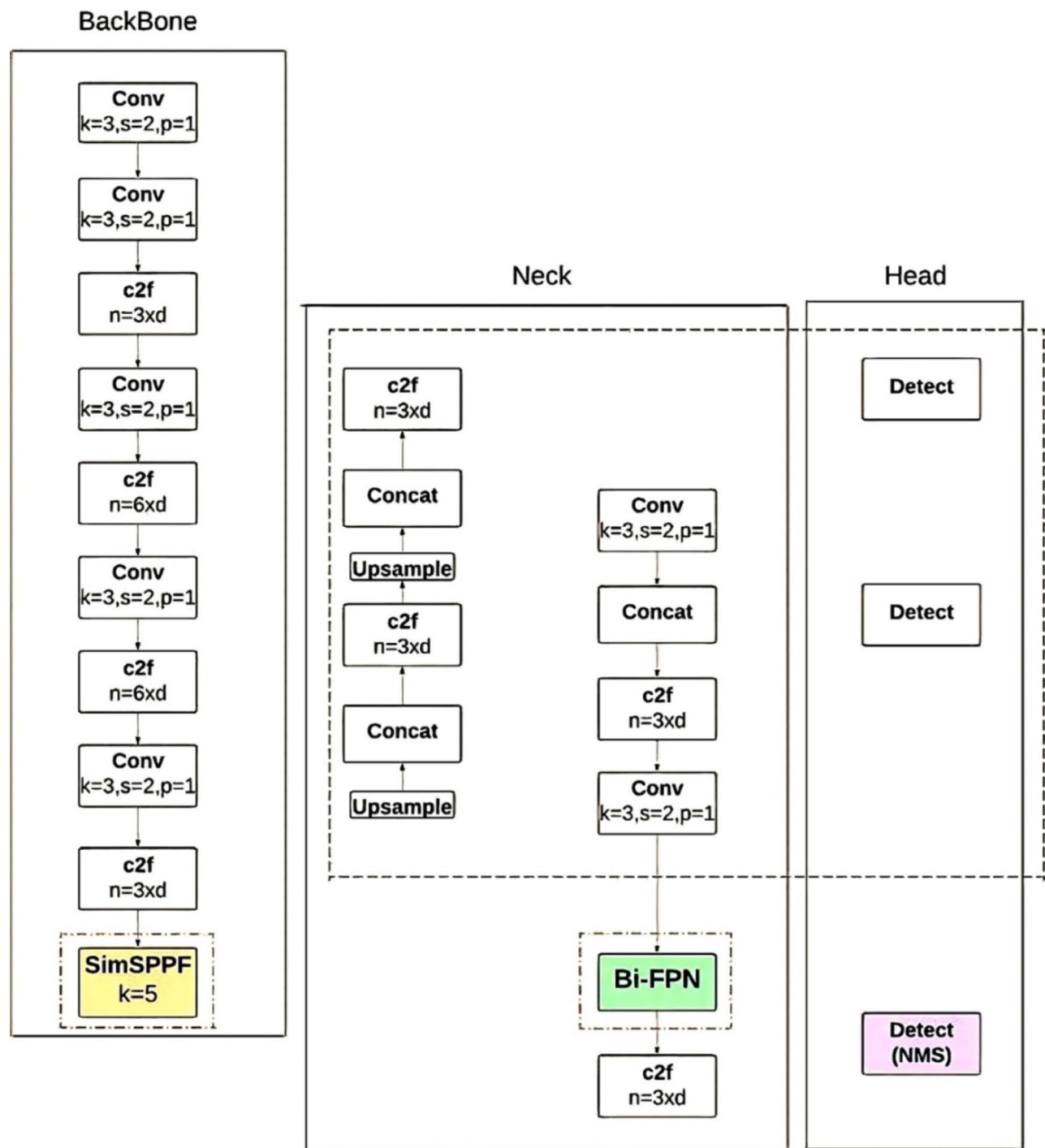


Fig. 4. Enhanced YOLOV8.

where I represent the input images in the above Eq. (4).

Advanced pooling operations are performed over the extracted features to enhance the resilience of the model. Pooling operations helps in effectively handling the scale variations by performing the pooling the extracted features at several layers of the backbone to capture all the small scale and the large-scale features. When a person falls closer to the camera, they will appear larger in size and if they fall far away from the camera, they will appear larger in size. Adaptive pooling operations helps in reducing the sensitivity to such variations by aggregating features over larger areas of the image and helping the network focus on the most relevant features. This ensures that the model can effectively detect a fall regardless of whether the person appears larger or smaller due to variations in distance, camera angle, or orientation.

Enhanced feature fusion using BiFPN's adaptable weights

Traditional YOLOV8 uses conventional up sampling and down sampling process in the neck region for processing the input image. Though down sampling helps to reduce the spatial size of the input image, it reduces the precision in the boundary of the detected objects and as a result the smaller features goes unnoticed (i.e.) the fall occurrences happening far away from the camera goes undetected leading to inconsistent detections. Up sampling process restores only the spatial information but not the semantic depth leading to the generation of the false positives where normal movements are mistaken as fall events.

The proposed work optimizes these setbacks by introducing the Bidirectional Feature Pyramid Networks (BiFPN) in the neck of the YOLOV8 architecture as shown in Fig. 3. BiFPN iterates through the weighted feature fusions multiple times in both top down and bottom up directions to enable detection across multiple scales as follows

$$F_{topdown}^{t+1} = \sum_{i=1}^N \alpha_i^{top} F_i^t \quad (5)$$

where, t represents the iteration number (since BiFPN refines features multiple times), α_i is the learnable weight that adjusts how much importance each feature level gets.

In the next bottom up pass the refined features are updates as

$$F_{bottomup}^{t+1} = \sum_{i=1}^N \alpha_i^{bottom} F_{topdown}^{t+1} \quad (6)$$

This ensures that the smaller irrelevant features are removed while assigning higher weights to the most relevant features. In this way the proposed work ensures that both distant falls and closer falls are detected precisely without any miss. Thus incorporation of Bidirectional Feature Pyramid Networks (BiFPN) in the neck region of the YOLOV8 helps in increasing the detection accuracy even when there is a very minute change in the detection environment. BiFPN uses learnable weights to combine features from different levels. This adaptive weighting ensures that more relevant features are retained and the irrelevant background noises are removed. The weighted feature fusions are represented as

$$F_{(fusion)} = \sum_i w_i * x_i \quad (7)$$

where, $F_{(fusion)}$ represents the features to be fused, w_i denotes the feature maps from each level and x_i denotes the adaptable weights.

ReLU activation function for faster convergence

The proposed approach replaces the conventional SiLU activation with the ReLU activation function. SiLU has a tendency to saturate near the larger negative values thereby producing very small negative gradients, slowing down the learning process. This leads to a slow convergence process while detecting the certain edge-case falls. Whereas, ReLU maintains a stronger gradient (1 for positive values), ensuring weight updates remain significant and effective by the following consideration. ReLU helps in addressing the vanishing gradient problems allowing for deeper convergence inside the YOLOV8 network. ReLU is also computationally simple which makes it very much suitable for the real time application like fall detection where the minimal inference time is considered as the higher priority.

If the input is positive ReLU will output the same input value. If the input is less than zero, ReLU will output zero and this process is back propagated through the network for all the possible positive values to overcome the vanishing gradient problem. ReLU activation function ignores minute negative values which primarily represents the noise in the low-resolution images thereby leading to enhanced accuracy. Gradients calculated during the process will back propagate and update network weights throughout.

If 'x' denotes the input then the ReLU activation function¹⁰ is mathematically denoted as

$$f(x) = \max(0, x) \quad (8)$$

$$ReLU(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases} \quad (9)$$

As per the above Eq. (9) for all the positive inputs the weight updation remains strong and stable eliminating the risks of vanishing gradients since the gradient here is a non zero component. This positive gradient fastens the learning process and it enables the network to learn all the diverse fall patterns effectively. The faster convergence of the ReLU activation function is shown in the Eq. (10)

$$W^{(t+1)} = W^{(t)} - \eta * \frac{\partial L}{\partial W} \quad (10)$$

where $W^{(t)}$ is the weight at time step t , η represents the learning rate, $\frac{\partial L}{\partial W}$ gives the gradient of the loss function L with respect to weight W . Replacing the SiLU activation function with ReLU speeds up weight updates by keeping gradients strong and eliminating larger computations thereby making the proposed YOLOV8 computationally efficient while implementing in the real time edge devices.

Post processing for removing redundant detections

Although YOLOV8 follows anchor free detection, it generates redundant/overlapping boundary boxes for the same person leading to multiple false alarms. To mitigate this redundancy the proposed approach incorporates No-Maximum Suppression¹² as the necessary post processing step. During the NMS process the generated bounding boxes are sorted based on their confidence scores. The confidence score is calculated as

$$\text{Confidence_value} = \text{Object_Score} * \text{ClassScore} \quad (11)$$

where object_Score denotes the likelihood of the fallen person inside the predicted bounding box and Class_Score represents the confidence of the model in predicting the fallen person. Once the Confident values are calculated the bounding box with the highest confidence score is added to the selected list. The Intersection Over Union value¹⁵ is calculated between the chosen bounding box and the remaining box as follows

$$\text{IoU} = \text{Area of_Intersection} / \text{Area of_Union} \quad (12)$$

The IoU values beyond the chosen threshold are considered as the overlapping bounding boxes and they are eliminated. This process is repeated until all the redundant bounding boxes are removed. Thus, by enhancing the existing YOLOV8 with SimSPPF, BiFPN, ReLU activation and NMS helps in faster and precise predictions across multi scale variations. Yet YOLOV8 is prone to single model errors in some ambiguous body postures where a normal activity may be mistaken as a fall incident. To overcome this shortfall the next part of the proposed work considers the predictions of RetinaNet and DETR together in the ensemble phase.

Retinanet for fall detection

In real-time fall detection datasets, there is a significant imbalance between classes (i.e.) the number of fall incidents is substantially lower than that of non-fall incidents. This disparity arises because falls are relatively rare events compared to routine daily activities. Consequently, the data collected for fall detection is heavily skewed towards non-fall events, leading to a class imbalance. This imbalance poses challenges for machine learning models, as they may become biased towards predicting the majority class, thereby reducing the accuracy of fall detection. RetinaNet effectively addresses this problem with the help of the focal loss function¹⁶. Further this RetinaNet performs object detection in one forward pass leading to faster inference times than the two stage detectors. Henceforth RetinaNet is chosen as one of the fall detection model in the proposed approach.

RetinaNet's focal loss function assigns lesser weights to the more frequent classes and higher weights to the lesser number of instances in order to make a strong emphasis on the falls. By prioritizing losses from difficult-to-detect falls, the model adapts itself to better recognize these rare but crucial events.

$$\text{FL}_{(pt)} = -\alpha_t (1 - pt)^\gamma \log(pt) \quad (13)$$

where pt is the predicted probability of the true class. α_t is a scaling factor for the class. γ is the focusing parameter that adjusts the rate of down-weighting easy examples.

RetinaNet uses anchor boxes² to classify potential fall regions. These transparent boxes are drawn around predicted fall areas, and the model checks them to ensure all objects are covered. If an anchor box closely matches the fall's shape and size, the model confirms the fall. This method is effective in crowded areas, where RetinaNet adjusts anchor boxes to locate falls, even when the fallen persons are partially occluded in the crowded regions. The IoU (Intersection over Union) measures overlap between anchor boxes, with higher IoU indicating a higher fall probability.

Enhanced RetinaNet with RESNET 'D' as the backbone

Figure 5 shows the proposed RetinaNet architecture with ResNet D as the backbone incorporated with the Feature Pyramid Networks followed by the classification and regression subnets.

The traditional RetinaNet uses ResNet 50 for feature extraction, which performs well on high-resolution images but struggles with low-resolution ones, leading to generation of false positives. To address this, the proposed model replaces ResNet 50 with ResNet D¹⁸, which anti alias pooling operations instead of 1×1 convolutions, ensuring no information is missed during feature extraction. The alias pooling operations are elaborated in the Eqs. (14, 15)

ResNet D makes use of low pass Gaussian Kernel filters followed by standard pooling operations as

$$x' = G * x \quad (14)$$

x represents the original input image, G denotes the Gaussian Kernel filter, $*$ stands for the convolutional operation, x' symbolizes the resultant feature maps after the standard convolutions.

The summation of the convolutions are obtained by

$$y(i, j) = \sum_m \sum_n x'(i + s.m, j + s.n) . w(m, n) \quad (15)$$

S is the stride of the convolution, indicating the step size. $w(m, n)$ represents the weight of the kernel at position (m, n) . Thus this anti alias pooling in the ResNet D block establish a better retention of the features in the low resolution images. Additionally, the Feature Pyramid Network (FPN)⁸ in RetinaNet handles multi-scale variations, capturing both small and large features to ensure falls are detected regardless of position or size.

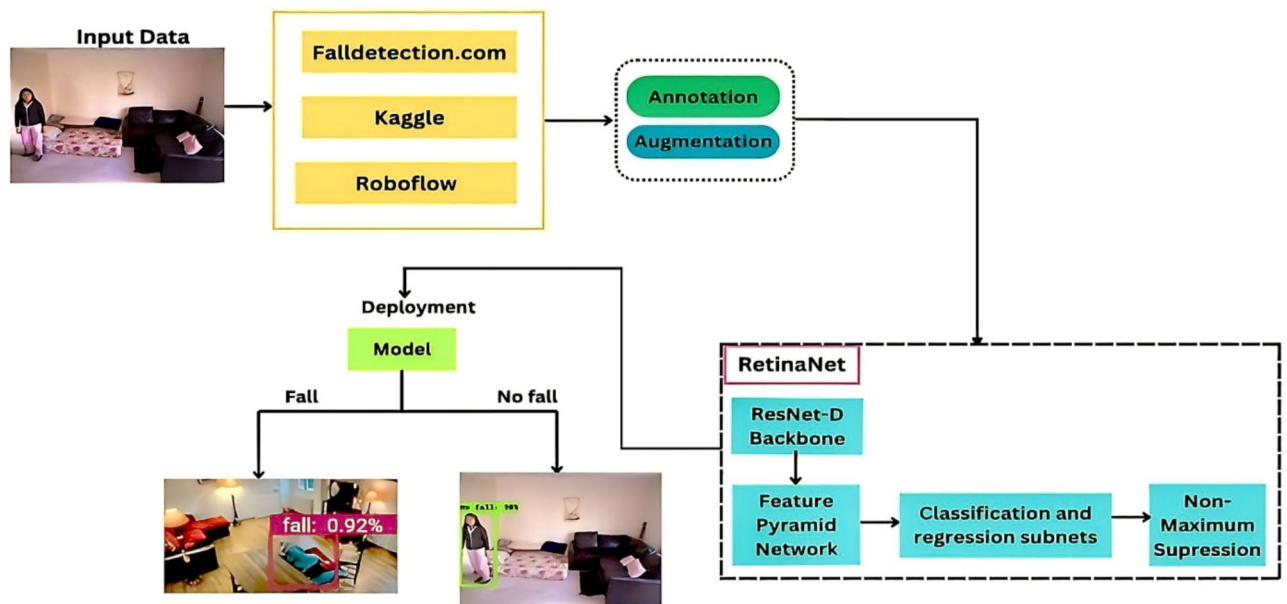


Fig. 5. Proposed RetinaNet.

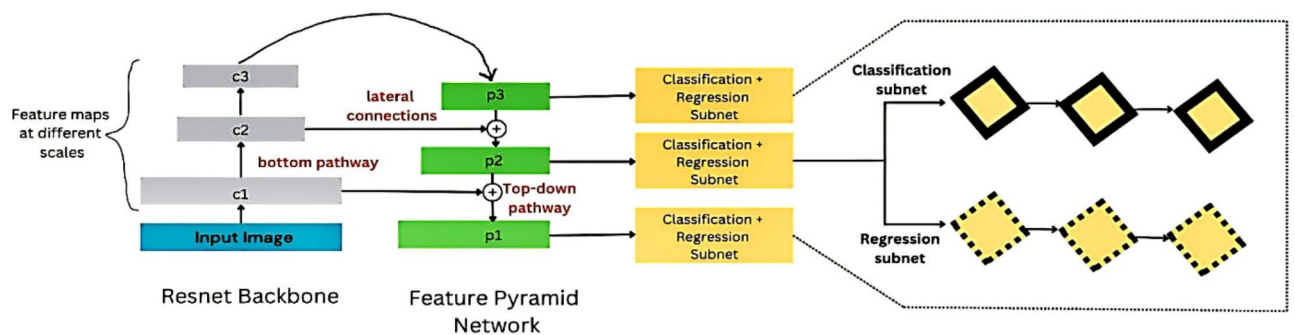


Fig. 6. RetinaNet regression subnets.

Classification subnets for enhancing the accuracy

The role of the classification subnet is to reduce the false positive rates by ensuring that only genuine falls are predicted. Initially, the RetinaNet will make use of the Feature Pyramid Networks to generate the anchor boxes and perform a detailed feature extraction throughout the image. These features will provide finer and intricate details of all the objects inside each and every anchor boxes. The classification subnets will consider these detailed features at distinct scales to classify the anchor boxes into two distinct classes as 'fall' and 'no fall' respectively. Classification subnets will ensure that the system doesn't mistake real fall events with other types of movements or daily day activities thereby it helps in increasing the overall reliability of the model.

Box regression subnets

The role of the box regression subnet is to fine tune the shape and size of the anchor boxes around the fallen person(s) in the image so that the anchor boxes are in best fit with the ground truth values. Overall, the classification subnet and box regression subnet help in increasing the accuracy of the model. It makes sure that the fallen persons are located precisely within the input image. Box Regression subnet is mathematically represented as

$$O = \text{Convolution}(F') \quad (16)$$

where O is the output feature map. Convolution represents the final convolutional prediction.

F' denotes the feature maps passing through the convolutional layers.

As indicated in the Fig. 6 C1, C2 and C3 are feature maps at different scales. The backbone is bottom-up path and the scale decreases as the feature extraction stage ascends. P1, P2, P3 are the up sampled semantics that are produced from the lateral connection with the bottom-up path. Classification and regression subnet share

two independent branches that run concurrently, one to predict the object classification score and the other to compute the bounding box offsets of each anchor box at different scales.

Detection transformers (DETR) for fall detection

DETR²⁵ is a transformer-based model that combines Convolutional Neural Networks (CNNs) with transformers to interpret the entire image rather than specific points. Its architecture uses CNNs for feature extraction, detecting edges and textures to generate feature maps. These maps are then processed by the transformer encoder and decoder for a deeper understanding of the scene. By considering the full image, DETR is particularly effective in crowded areas with complex backgrounds. It can distinguish between a person lying down and one who has fallen, focusing only on relevant image parts, making predictions more reliable.

Self attention mechanism

DETR's multi head self-attention mechanism⁶ helps in precisely predicting the falls in the crowded images. This feature considers each and every pixel of the input image and compute a weighted sum of these images. The entire image is divided into grids. Each and every grid will contain specific information pertaining to the image and these specific data are considered as features to that particular image. These multiple heads deployed in the DETR model will look at the grids at different angles for a better understanding and it will convert these observed features into distinct feature vectors namely Queries (Q_i) which represent specific queries in each frame, such as identifying standing persons or seated individuals., Keys (K_i) It encompasses answers to the Queries (Q) and Values (V_i) provides the elaborate information regarding the individuals in the frame. Linear Projections of the transformer model will break the image and convert them into different forms to establish a deeper and multi-dimensional understanding of the image. If X_i represents the input feature vector from the CNN backbone the vectors²¹ Q_i , K_i and V_i are calculated from the linear projection as

$$\text{Query_Projection} = x_i * w_q \quad (17)$$

where w_q is the weight matrix calculated from the input images.

$$\text{Key_Projection} = x_i * w_k \quad (18)$$

where w_k is a learned filter that helps in focusing the different parts of the input image.

$$\text{Value_Projection} = x_i * v_i \quad (19)$$

where V_i is the value_projection which helps in organizing the raw pixels into the predictable form. The final attention score²¹ is computed using the dot product operation as

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{(QK^T)}{\sqrt{d_k}} \right) V \quad (20)$$

DETR encoder

DETR encoder is responsible for high level feature extraction. DETR's encoder will take the input image and passes it to the series of convolutional layers. Initially the input image is passed through the series of the convolutional neural networks and visual feature extraction is done and it is represented as FCNN. These convolutional neural networks will add positional encoding to the extracted features to provide high level spatial information and it is represented as FP_E . This is given as an input to the transformer encoder layers where in depth global understanding is made. The relationship between the person who has fallen and the surroundings around the person is better understood for precise classification. The final output feature map is represented as $FDETR$ ²².

$$FCNN(\text{PositionalEncoding}) \rightarrow FP_E(\text{TransformerEncoderLayers}) \rightarrow FDETR \quad (21)$$

BiPartite matching

In real time, an input may contain multiple objects leading to multiple predictions leading to the generation of false positives. The bipartite matching in DETR is a one to one matching technique which involves in associating the predicted bounding boxes with ground truth objects in an image. This one to one matching technique helps in eliminating the overlapped predictions in the crowded images where the system might generate several overlapping predictions which will be considered as a separate fall events leading to increased number of false positives. This one to one matching will ensure that each prediction is exactly matched with only one relevant ground truth labels leading to reduced redundancy. Results have shown that this mindful matching reduced the total number of generated false positives in the crowded environments.

Working of bipartite matching

DETR generates bounding boxes, class labels, and confidence scores for each object, while ground truths are annotated with bounding boxes and labels. A bipartite graph is constructed between predictions and ground truths, with connections describing possible matches. The match is verified using a cost matrix for bounding box similarity and label accuracy. The Hungarian algorithm²⁶ ensures one-to-one matching between predictions and ground truth objects, minimizing matching costs. DETR has shown accurate results even with images taken

from diverse angles. The following (Table 2) represents the Bipartite matching table where each row corresponds to the ground truths and each column corresponds to the predicted Falls.

Ensemble learning

Ensemble learning is a technique in which multiple distinct models are trained over the same task and one final decision is taken from the predicted results using voting or the averaging techniques. The proposed methodology ensembles three models namely modified YOLOV8, modified RetinaNet and DETR and the final result is predicted by majority voting technique³.

The input images are processed independently by the chosen three models and three individual outputs (Fall/No-Fall) decisions are derived. These three single model outputs are given as an input to the ensemble model based on majority voting. Let M1, M2 and M3 denotes the individual outputs of the model as M1 representing the YOLOv8 prediction result, M2 symbolizing the RetinaNet prediction result and M3 explaining the DETR prediction result. Each model outputs a binary classification as “Mi ∈ {0,1}, where 0 = No Fall, 1 = Fall for every iterations i = 1,2,3. These three single model outputs are given as an input to the ensemble model based on majority voting.

The final prediction Y is determined using majority voting as shown in the Eq. (22)

Y = { 1, if ∑_{i=1}^3 M_i ≥ 2(atleasttwomodelspredictsapossiblefall) 0, Otherwise (22)

If two or more models predict the output as “Fall” (ΣMi ≥ 2), the final decision will be given as Fall (i.e. Y = 1), if fewer than two models predict “Fall” (ΣMi < 2), the final decision will be No Fall (i.e. Y = 0).

Ensemble technique helps to overcome the over fitting problem as the final prediction is not dependent on one single model. False positives are reduced to a greater extent as if one model fail to capture the fall, the other model's prediction results are also considered before the final decision thereby ensuring that almost all the falls are detected by the system without any miss.

In the proposed work, the chosen three models will make its own prediction for the input data as shown in the Fig. 7. Each models' decision will be either a 'Fall' or a 'No Fall' and these will be counted as a vote. Then the class with the higher number of votes is selected as the final and majority decision by the model. If a tie has occurred between the predictions, the ensemble technique will predict the result based on the weighted voting method where each model is assigned some weights based on its performance. The model with the highest weight is given more priority in case of a tie situation.

Table 3 shows the results of the soft and hard voting instances. In the soft voting instances, the average of the class probability by all the classifiers are calculated and the class with the highest average probability will be selected for the final decision. From the Table 3 It is observed that the average class probability for 'Fall' class is higher than the 'no Fall' class and the final decision will be given as a 'FALL' which turns out to be a true positive result for the above given input. In the hard voting instances, each and every models individual classification results are retrieved and voted. The class with the maximum number of votes will be considered as the final decision. In the Table 3 'Fall' class has the maximum number of votes and the final decision is made as a fall and it is a true positive result.

Timely alerts to the care takers

Figure 8 shows the real time fall alert issued to the concerned care takers via the Twilio platform, a cloud communication platform that offers APIs for sending messages and making calls. Whenever a fall is detected, an alert message embedded with information of the care takers details and the picture of the fallen person is sent to the Twilio Application Programming Interface.

Upon completing the necessary verification, the proposed method will communicate the real time alert notification through the SMS mode thereby ensuring timely delivery of information about the fall.

Experimental result analysis

The proposed YORD fall detection model is implemented in the Google colab environment with the help libraries such as PyTorch, TensorFlow and ultralytics for model training, evaluation, and analysis. To improvise the dataset, augmentations were used along with Albumentations to mimic realistic variations in the data, making the model more resilient in real time. Standard performance parameters such as the mean Average Precision (mAP), Precision, Recall, and F1 Score are considered to indicate the detection accuracy and reliability. To compare YORD with existing methods, we evaluated YORD's performance against the single model methods. The results prove that the proposed YORD outperforms the independent models at detecting falls under various

GroundTruth\Predicted	PredFall1	PredNoFall1	PredFall2	PredNoFall2
GroundTruthFall1	Cost(1,1)	Cost(1,2)	Cost(1,3)	Cost(1,4)
GroundTruthNoFall1	Cost(2,1)	Cost(2,2)	Cost(2,3)	Cost(2,4)
GroundTruthFall2	Cost(3,1)	Cost(3,2)	Cost(3,3)	Cost(3,4)
GroundTruthNoFall2	Cost(4,1)	Cost(4,2)	Cost(4,3)	Cost(4,4)

Table 2. Bipartite matching table for fall and no-fall detection.

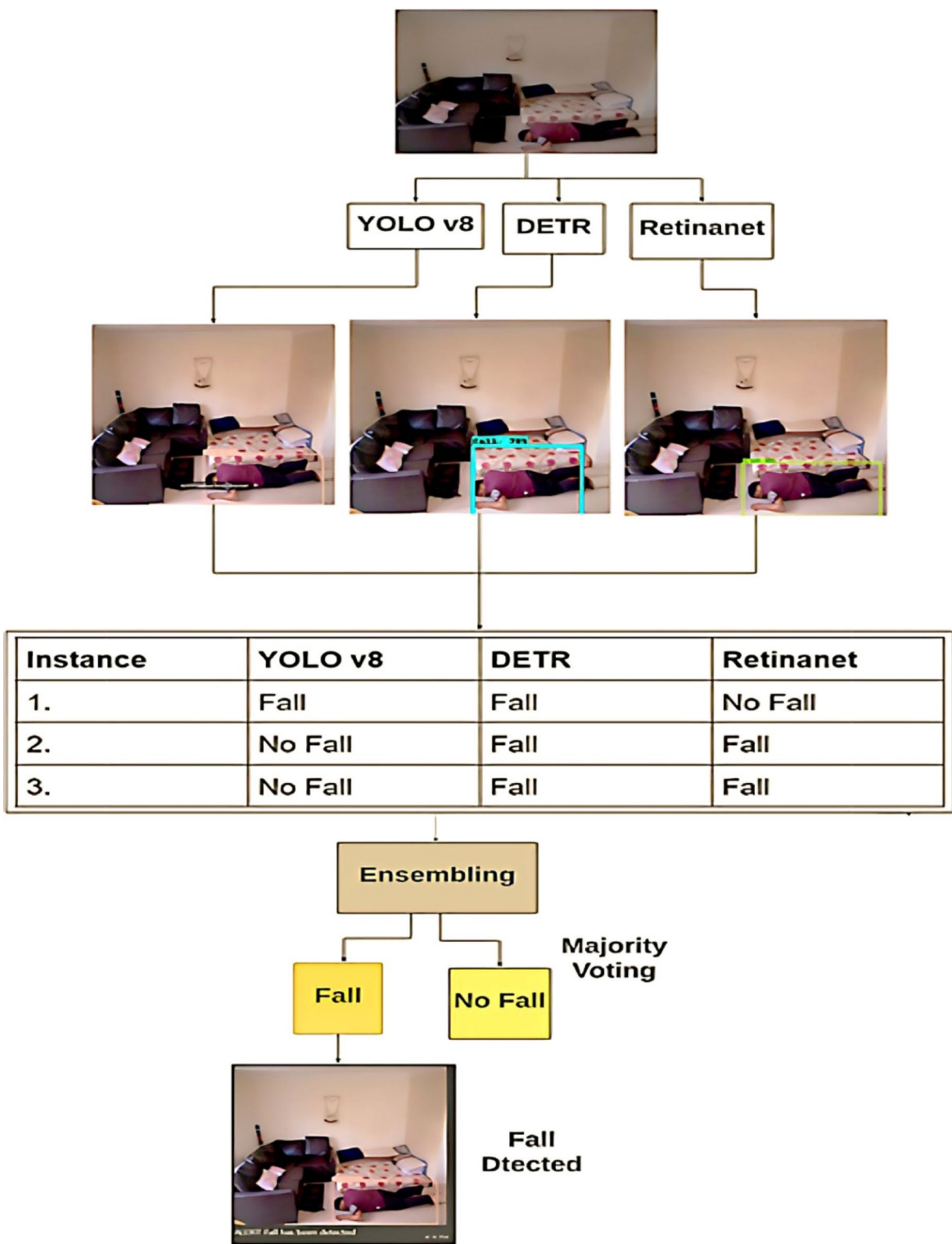


Fig. 7. Ensemble results.

challenging environments such as crowded regions, low resolution images and scale variances. Table 4 depicts the simulation parameter setup.

EnhancedYOLOV8 result analysis

Figure 9 portrays the performance analysis of the YOLOV8 model. The 'X' axis represents the epochs and the 'Y' axis represent the training loss/mean average precision. From the results it can be observed that the models box loss and class loss curves are decreasing over the iterations which means that the model's error rate is getting

Soft voting instances					
Instance number	Class Label	YOLOV8	RETINANET	DETR	Average Probability
1	FALL	0.925	0.89	0.87	0.895
2	NO FALL	0.005	0.74	0.002	0.249
Hard voting instances					
Instance number	Class Label	YOLOV8	RETINANET	DETR	FINALDECISION
1	FALL	FALL	NO FALL	FALL	FALL
2	FALL	NO FALL	FALL	FALL	FALL
3	FALL	NO FALL	FALL	FALL	FALL

Table 3. Soft and hard voting instances.

decreased and it is becoming more accurate in predicting the results. It is also observed that the validation loss is also continually decreasing over the epochs which gradually reduces the problem of over fitting as the model is able to accurately predict the fall and no fall results over the new unseen data as well.

The above (Table 5) depicts the performance of the proposed YOLOV8. From the table it is observed that the proposed v8's metrics namely mAP, precision and recall are comparatively higher than the existing version. This proves the improvement in the performance of the proposed system. It is observed that the proposed model is more robust as it is able to predict the falls in diverse conditions like low resolutions, crowded environments and when the person is falling far away from the camera.

Table 6 projects the loss values of the proposed model. Results prove that the box loss, object loss and class loss of the proposed model are comparatively lesser than the existing model. Box loss will evaluate the accuracy of the position and the size of the detected objects. Object loss will evaluate the presence of the persons inside the detected bounding boxes and class loss will assess the accuracy of the model's class predictions by comparing the predicted class labels with the actual class labels of the detected objects. Experimental results shows that the predicted loss values are comparatively lesser than the existing models which indicates the improved performance.

From the above (Table 7) it is observed that the proposed YOLOV8 response time is reduced to a greater extent without compromising the accuracy. This is achieved by removing the smaller detection heads in the proposed model. These smaller detection heads are responsible for predicting the extremely smaller objects. This smaller detection head removal decreased the response time which means that the real time alerts are issued instantly as soon as the falls are detected. Further the smaller detection heads removal will not compromise the accuracy in detecting the smaller objects when they fall far away from the camera as the model incorporates Bidirectional Feature Pyramid Networks in the backbone network which will traverse in both the top down and bottom up directions for handling the objects in different scales.

Enhanced RetinaNet result analysis

The loss function performance comparison for the proposed and existing RetinaNet models, as shown in Table 8, highlights significant improvements with the proposed RetinaNet using ResNet-D as the backbone. The box loss and class loss for the proposed model (0.0017 and 0.17, respectively) are substantially lower than those of the existing model (0.0045 and 1.23). Additionally, the validation loss and training loss of the proposed model are reduced to 1.7243 and 1.8035, compared to 2.0061 and 2.0498 for the existing model. These results proves the effectiveness of the proposed modifications in enhancing model precision and reducing error rates.

From the above (Table 9) it is observed that the proposed RetinaNet model is more robust than the existing RetinaNet as it is able to identify the falls precisely in various real time scenarios like crowded places, low resolution images and when the person is falling far away from the camera appearing to be smaller in size and when the person is falling closer to the camera and appears to be larger in size.

DETR result analysis

From the (Table 10) it is observed that the DETR model has achieved a perfect recall of one which symbolizes that the model is able to capture all the true positives without missing any actual falls. However, there is a possibility that the model may generate false positives, misinterpreting a "no fall" as a "fall," leading to false alarms. To handle this set back the results of the other two models namely enhanced YOLOV8 and enhanced RetinaNet are also considered by the majority voting technique of the ensemble model to conclude the final decision as a 'Fall'/'No Fall' respectively.

Table 11 explains about the loss values of the DETR model, for training and validation process. The overall training loss is at 0.35, which is satisfactory enough to say that the model learned efficiently. Validation loss, while slightly higher at 0.54, has a hue of over fitting, or certain challenges in data prediction. The bounding box loss for the training set is at 0.02 and for the validation set is at 0.05 which are considerably low and it indicates that the model exhibits a high precision in locating objects. Similarly, the classification error loss is almost negligible with the training value of 0.0001 and validation value of 0.009, which indicates an error-free precise labelling of the image. This proves the efficiency of the transformer model in classifying the fall and no fall activities.

As depicted in the Table 12, The YORD ensemble model surpasses traditional ensemble-based fall detection methods in terms of complexity, performance, and practical applicability by addressing the key limitations of

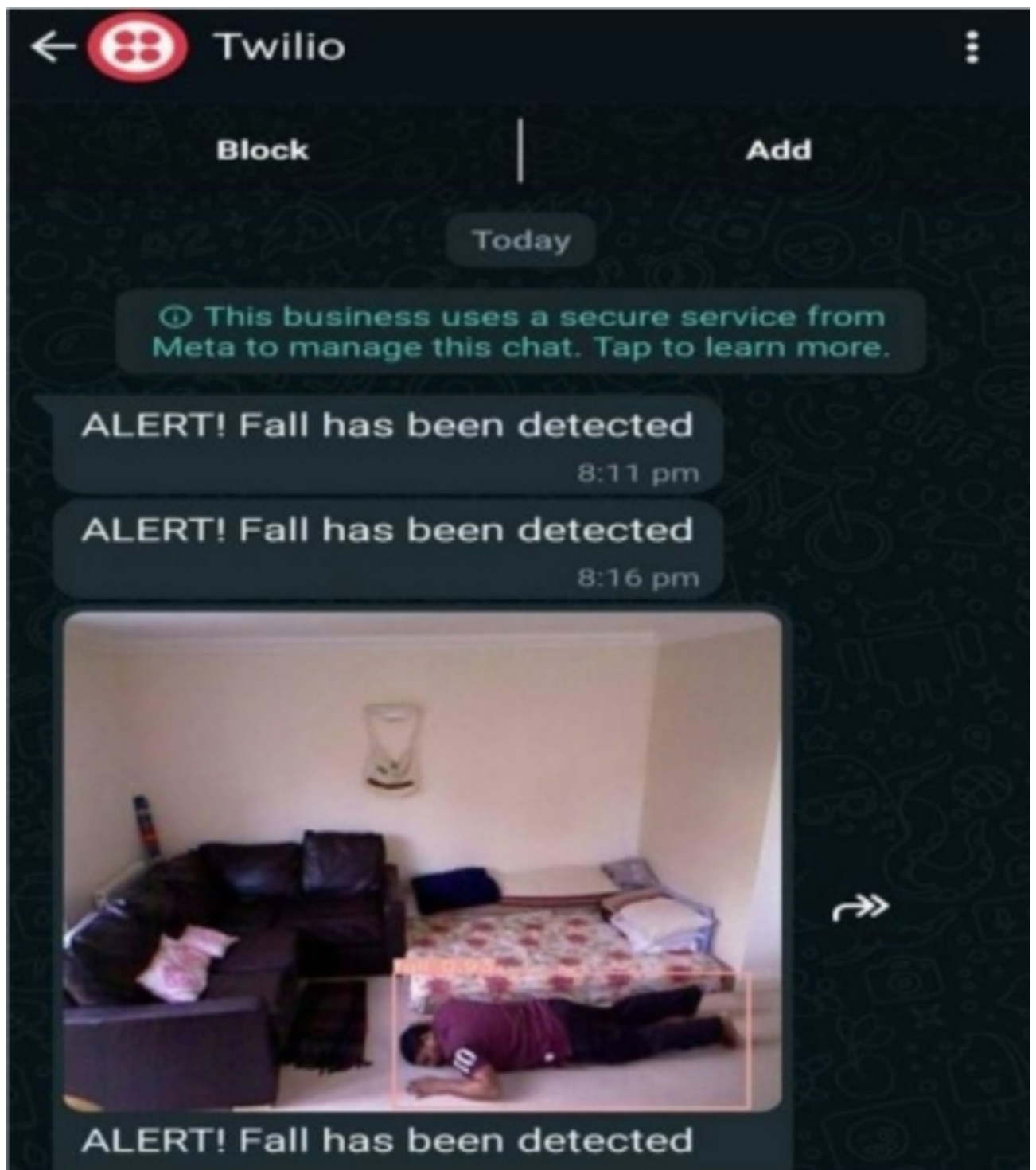


Fig. 8. Real time alerts via twilio platform.

existing approaches. Unlike conventional stacking or boosting-based ensembles, which require additional meta-classifiers and increase computational overhead, YORD utilizes a majority voting mechanism that integrates predictions from YOLO V8, RetinaNet, and DETR without requiring extra training layers. This reduces processing complexity while maintaining high detection accuracy. Traditional ensemble methods often rely on sequential execution, where multiple models process the input one after another, leading to increased latency. In contrast, YORD executes all three models in parallel, leveraging Google Colab's GPU-based processing to minimize inference time to just 0.39 s—making it more suitable for real-time applications. Performance-wise, YORD achieves an mAP (50–95) of 84.2%, outperforming individual models such as enhanced YOLO V8 (80.3%), RetinaNet (80.4%), and DETR (80.3%). This improvement is due to its ability to effectively handle

Parameter	Explanation
Dataset	Augmented fall detection dataset
Image resolution	640 × 480
YOLO V8 backbone	BiFPN and SimSPPF
RetinaNet backbone	ResNet D
DETR model	Transformer encoder/decoder (Baseline)
Confidence threshold	0.5
Voting mechanism	Majority voting with weighted tie-breaking
Processing platform	Google Colab
Implementation framework	PyTorch, TensorFlow
Augmentation	Albumentations

Table 4. Simulation parameter setup table.

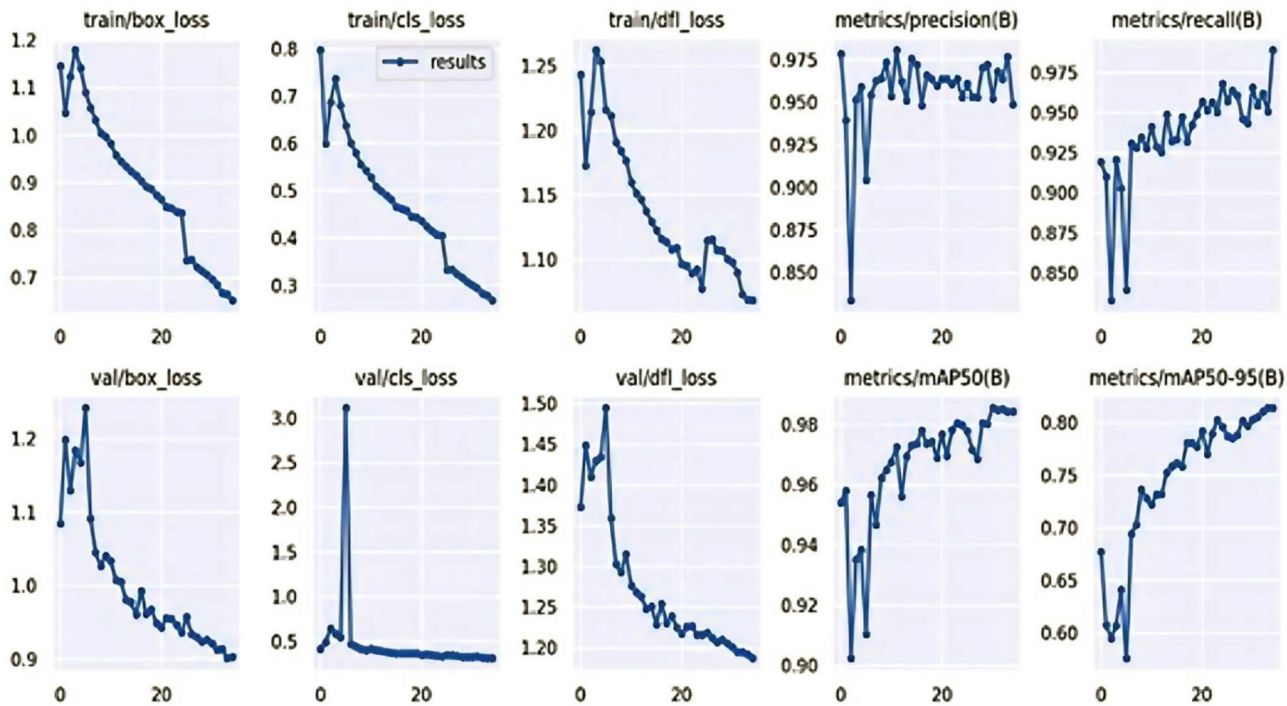


Fig. 9. YOLOV8 loss and precision curves.

low-resolution images, multi-scale variations, and crowded environments, where single-model approaches often struggle. Unlike traditional CNN-based fall detection methods that suffer from high false positives and difficulty detecting small-scale falls, YORD enhances robustness by integrating YOLO V8’s BiFPN for multi-scale feature extraction, RetinaNet’s Focal Loss for addressing class imbalance, and DETR’s transformer-based bipartite matching to remove redundant detections in cluttered scenes. Additionally, compared to sensor-based and WiFi-based fall detection techniques, which require specialized hardware and face privacy concerns, YORD operates purely on computer vision techniques, making it more scalable and easier to deploy in healthcare monitoring systems. The Twilio-based real-time alert mechanism further enhances its practical applicability, ensuring immediate notification to caregivers upon fall detection. Overall, YORD’s optimized computational efficiency, high detection accuracy, and real-time feasibility make it superior to traditional ensemble methods and an ideal solution for fall detection in real-world healthcare applications.

Although ensembling multiple models typically increases computational complexity, YORD mitigates this overhead through parallel execution, reduced redundant computations, and strategic model enhancements. Unlike traditional sequential processing, where each model runs independently, YORD executes all three models concurrently using parallel processing on Google Colab’s GPU-based infrastructure, significantly cutting down inference time. Additionally, redundant computations are minimized—for instance, DETR’s bipartite matching eliminates overlapping detections, reducing unnecessary processing, while RetinaNet’s focal loss prioritizes difficult detections, optimizing computational efficiency. Moreover, YOLO V8’s smaller detection heads were removed, reducing its memory footprint and accelerating inference speed. These optimizations, combined with

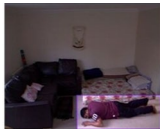
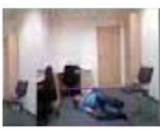
	Single person in frame		Crowded environment		Smaller in size		Low resolution		Multiple persons	
Input Images										
										
Evaluation metrics of existing and proposed YOLOV8										
	Existing	Proposed	Existing	Prop osed	Existing	Proposed	Existing	Proposed	Existing	Proposed
mAP 50–90	75.4	80.3	76.2	81.1	75.3	80.8	76.2	81.5	75.2	81.3
mAP 50	97.6	98.4	97.2	98.2	97.3	97.8	97.1	98.3	97.5	98.5
Precision	94.2	97.7	95.6	98.1	94.7	97.2	94.3	97.7	95.6	97.3
Recall	93.2	95	93.5	94.3	94.2	95.2	94.1	95.2	94.6	95.3
F1 Score	93.7	96.3	94.5	96.3	94.4	96.4	94.2	96.4	95.1	96.3

Table 5. Performance Analysis of enhanced YOLOV8.

Dataset samples	Box loss	Object loss	Class loss
ExistingYOLOV8	0.0948	0.0251	0.0054
ProposedYOLOV8	0.0664	0.0098	0.0012

Table 6. Loss function of enhanced YOLOV8.

Instance number	ExistingYOLOV8		ProposedYOLOV8	
	Accuracy (%)	Time (s)	Accuracy (%)	Time (s)
Instance 1	95.4	0.480	98.1	0.398
Instance 2	94.3	0.482	98.2	0.387
Instance 3	94.3	0.482	98.2	0.387

Table 7. Improved response time in the proposed YOLOV8.

Models	Box loss	Class loss	Validation loss	Training loss
Existing RetinaNet with Backbone as ResNet50	0.0045	1.23	2.0498	2.0061
Proposed Retina Netwith Backbone as ResNet-D	0.0017	0.17	1.8035	1.7243

Table 8. Loss function of the proposed Retinanet model.

Google Colab's cloud-based GPU acceleration, adaptive batch inference, and optimized tensor operations, allow YORD to maintain high accuracy with lower processing times, making it a real-time viable solution despite ensemble complexity. The proposed YORD is computationally more efficient and lightweight as it requires only counting predictions from each model. Unlike stacking or boosting, there is no need for additional training of meta-classifiers, making it suitable for real-time applications. Thus, majority voting balances complexity and speed, making it a practical choice for real-time fall detection. However, in real time parallel GPU execution will be the best suitable choice.

Ensembling multiple models may seem to increase the computational overhead but in the proposed approach the three different models are optimally chosen in such a way that the overall response time is kept as minimum as possible. The very first model YOLOV8 is a single stage detector and it is optimized for real time processing as it runs significantly faster than the other two models. The ensemble relies on parallel inference, where all three models process the same image simultaneously instead of sequentially. This minimizes the waiting time for each model's output. Since majority voting is a lightweight operation, it does not significantly contribute to processing overhead.

While YORD integrates multiple models (YOLO V8, RetinaNet, and DETR), it strategically optimizes computational efficiency to balance performance and real-time applicability. The system leverages parallel

Sample input images	Existing RetinaNet with ResNet 50 as backbone			Proposed RetinaNet with ResNet D as backbone		
	AP 50–95	AP 50	Recall	AP 50–95	AP 50	Recall
	71.8	95.2	78.8	78.2	97.6	80.2
	73.5	96.0	79.1	80.4	98.1	81.5
	69.2	94.1	76.7	75.9	96.8	78.9
	65.4	91.5	74.2	72.1	95.0	76.3

Table 9. Modified RetinaNet Results.

Categories	Accuracy	Precision	Map	Recall
Single person (fall)	91	98.2	97.9	98.6
Single person (no fall)	91	98.2	97.9	98.6
Low resolution images	85	98.2	97.9	98.6

Table 10. DETR prediction results.

Categories	Loss values
Training loss	0.35
Validation loss	0.54
Training loss bounding box	0.02
Training loss classification error	0.0001
Validation loss bounding box	0.05
Validation loss classification error	0.009

Table 11. Loss values of the DETR model.

processing to reduce latency, ensuring that model inference occurs simultaneously rather than sequentially. Model pruning and optimizations further mitigate computational overhead—YOLO V8 removes smaller detection heads to improve efficiency, RetinaNet integrates ResNet-D for enhanced feature extraction without excessive resource consumption, and DETR’s bipartite matching reduces redundant predictions. Despite the ensemble approach, Table 12 shows that YORD maintains an average processing time of 0.39 s, which is comparable to standalone models like RetinaNet (0.40 s) and DETR (0.40 s), demonstrating that accuracy improvements outweigh the minor increase in computational demand. Additionally, hardware acceleration using TensorFlow, PyTorch, and GPU-based processing enhances real-time performance. Future implementations can further optimize YORD for edge devices through model quantization and batch inference, making it a highly practical and scalable solution for real-time fall detection in healthcare and surveillance applications.

Table 12 indicates that YORD maintains a lower average processing time than RetinaNet or DETR despite integrating three models, primarily due to parallel execution, model optimizations, and efficient inference

Environment/metric	YORD (Ensemble)	Enhanced YOLO V8	Enhanced RetinaNet	Baseline DETR
Crowded environments	82.30%	81.1%	75.9%	79.10%
Low-resolution images	84.70%	81.5%	72.10%	81.30%
Multiple scales	95.60%	93.48%	77.40%	80.60%
mAP (50–95)	84.20%	80.3%	80.40%	80.30%
Precision	97.40%	97.2%	96.8%	94.70%
Recall	95.20%	94.3%	81.5%	92.30%
F1 score	96.30%	96.3%	91.90%	93.40%
Average processing time	0.39 s	0.37 s	0.40 s	0.40 s

Table 12. Detection accuracy and performance metrics comparison.

strategies. Instead of processing each model sequentially, YORD runs YOLO V8, RetinaNet, and DETR in parallel, distributing computations across multiple CPU/GPU cores to reduce latency. Additionally, batch inference is employed, where multiple frames are processed simultaneously, improving throughput while minimizing redundant computational overhead. Model distillation techniques, such as removing redundant detection heads in YOLO V8, further enhance efficiency by streamlining inference without sacrificing accuracy. Hardware acceleration using GPUs in TensorFlow/PyTorch environments ensures faster computation times, enabling real-time fall detection. These combined optimizations allow YORD to maintain superior detection accuracy while keeping processing times lower than standalone RetinaNet or DETR models.

Conclusions and future work

In the proposed work, the incorporation of fine-tuned real time object detection techniques namely YOLOV8 with BiFPPN, RETINANET with RESNETD backbone and DETR enhances the detection accuracy and effectiveness of the detection systems. The proposed YORD ensemble model achieves notable performance improvements over single-model approaches in fall detection, particularly in challenging environments. Compared to individual models (YOLO V8, RetinaNet, and DETR), the YORD model improves mean Average Precision (mAP) by approximately 5–8% on average across scenarios. Precision and recall metrics show similar gains, with improvements of around 3–5% in precision and 4–6% in recall, especially in crowded and low-resolution environments where individual models struggle. Additionally, the YORD model reduces false positives by about 7–10% and false negatives by 5% on average compared to standalone models. These improvements indicate that YORD meets the performance requirements for reliable real-time fall detection. The YORD model surpasses traditional single-model approaches by effectively addressing the limitations of each individual detection model through a combined, ensemble-based approach. YOLO V8 provides speed and accuracy, RetinaNet addresses class imbalance with focal loss, and DETR’s transformer-based attention mechanism captures global context in complex or cluttered backgrounds. YORD’s ensemble design uses a majority voting system that leverages the strengths of each model, leading to more accurate fall detection with fewer false positives and false negatives. This research opens several new pathways for further advancements in fall detection and related applications. Future researchers can build on this work by exploring adaptive weighting mechanisms within the ensemble to dynamically prioritize model outputs based on environmental conditions, to optimize the detection accuracy. Another promising direction is to enhance the real-time performance through hybrid edge-cloud Processing techniques to potentially improve the processing speed while maintaining the accuracy. This research thus provides a robust framework and insight for future innovations in multi-model ensemble systems for highly accurate, real-time object detection in varied environments.

Data availability

The datasets generated during and/or analysed during the current study are not publicly available but are available from the corresponding author on reasonable request.

Received: 19 November 2024; Accepted: 7 April 2025
Published online: 26 September 2025

References

1. Rezaei, A. et al. Unobtrusive human fall detection system using mmwave radar and data driven methods. *IEEE Sens. J.* **23**(7), 7968–7976 (2023).
2. Qi, P., Chiaro, D. & Piccialli, F. FL-FD: Federated learning-based fall detection with multimodal data fusion. *Inform. Fus.* **99**, 101890 (2023).
3. Sheikh, S. Y. & Jilani, M. T. A ubiquitous wheelchair fall detection system using low-cost embedded inertial sensors and unsupervised one-class SVM. *J. Ambient. Intell. Humaniz. Comput.* **14**(1), 147–162 (2023).
4. Durga Bhavani, K. & Ferni Ukrit, M. Design of inception with deep convolutional neural network based fall detection and classification model. *Multimedia Tools Appl.* **83**(8), 23799–23817 (2024).
5. Wang, S. et al. Human motion recognition by a shoes-floor triboelectric nanogenerator and its application in fall detection. *Nano Energy* **108**, 108230 (2023).
6. Wang, P. et al. A convolution neural network approach for fall detection based on adaptive channel selection of UWB radar signals. *Neural Comput. Appl.* **35**(22), 15967–15980 (2023).
7. Bharathi, V., Monikavishnuvarthini, A., Dhakne, A. & Preethi, P. AI based elderly fall prediction system using wearable sensors: A smart home-care technology with IOT. *Meas. Sens* **25**, 100614 (2023).

8. Kulurkar, P. et al. AI based elderly fall prediction system using wearable sensors: A smart home-care technology with IOT. *Measurement Sens.* **25**, 100614 (2023).
9. Fula, V. & Moreno, P. Wrist-based fall detection: Towards generalization across datasets. *Sensors* **24**(5), 1679 (2024).
10. Noor, N. & Park, I. K. Factorized 3D-CNN for real-time fall detection and action recognition on embedded system. *IEEE Access*, (2024).
11. Maray, N., Ngu, A. H., Ni, J., Debnath, M. & Wang, L. Transfer learning on small datasets for improved fall detection. *Sensors* **23**(3), 1105 (2023).
12. Gao, M. et al. Fall detection based on OpenPose and MobileNetV2 network. *IET Image Proc.* **17**(3), 722–732 (2023).
13. Amsaprabhaa, M. et al. Multimodal spatiotemporal skeletal kinematic gait feature fusion for vision-based fall detection. *Expert Syst. Appl.* **212**, 118681 (2023).
14. Alanazi, T., Babutain, K. & Muhammad, G. A robust and automated vision-based human fall detection system using 3D multi-stream CNNs with an image fusion technique. *Appl. Sci.* **13**(12), 6916 (2023).
15. Mobsite, S., Alaoui, N., Boulmal, M. & Ghogho, M. Semantic segmentation-based system for fall detection and post-fall posture classification. *Eng. Appl. Artif. Intell.* **117**, 105616 (2023).
16. Li, Y., Zuo, Z. & Pan, J. Sensor-based fall detection using a combination model of a temporal convolutional network and a gated recurrent unit. *Futur. Gener. Comput. Syst.* **139**, 53–63 (2023).
17. Fei, K. et al. Flow-pose Net: An effective two-stream network for fall detection. *Vis. Comput.* **39**(6), 2305–2320 (2023).
18. Egawa, R., Miah, A. S. M., Hirooka, K., Tomioka, Y. & Shin, J. Dynamic fall detection using graph-based spatial temporal convolution and attention network. *Electronics* **12**(15), 3234 (2023).
19. Wang, Y. & Deng, T. Enhancing elderly care: Efficient and reliable real-time fall detection algorithm. *Digital Health* **10**, 20552076241233690 (2024).
20. Ergüder, H., Uzun, T. & Baday, M. Advancing fall detection utilizing skeletal joint image representation and deformable layers. *Image Anal. Stereol.* **43**(1), 97–107 (2024).
21. Li, B., Li, J. & Wang, P. Fall detection algorithm based on global and local feature extraction. *Pattern Recogn. Lett.* **185**, 31–37 (2024).
22. Wang, X., Talavera, E., Karastoyanova, D. & Azzopardi, G. Fall detection with a nonintrusive and first-person vision approach. *IEEE Sens. J.* **23**(22), 28304–28317 (2023).
23. Chu, Y., Cumanan, K., Sankarpandi, S. K., Smith, S. & Dobre, O. A. Deep learning-based fall detection using WiFi channel state information. *IEEE Access* **11**, 83763–83780 (2023).
24. De, A., Saha, A., Kumar, P. & Pal, G. Fall detection method based on spatio-temporal feature fusion using combined two-channel classification. *Multimedia Tools Appl.* **81**(18), 26081–26100 (2022).
25. Li, J., Gao, M., Wang, P. & Li, B. Fall detection algorithm based on pyramid network and feature fusion. *Evol. Syst.* **15**(5), 1957–1970 (2024).
26. Hnoohom, N., Mekruksavanich, S. & Jitpattanakul, A. Pre-impact and impact fall detection based on a multimodal sensor using a deep residual network. *Intell. Automat. Soft Comput.* **36**, 3 (2023).
27. Ha, T. V., Nguyen, H. M., Thanh, S. H. & Nguyen, B. T. Fall detection using mixtures of convolutional neural networks. *Multimedia Tools Appl.* **83**(6), 18091–18118 (2024).
28. Li, J. et al. KAMTFENet: A fall detection algorithm based on keypoint attention module and temporal feature extraction. *Int. J. Mach. Learn. Cybern.* **14**(5), 1831–1844 (2023).
29. Wang, S. & Wu, J. Patch-transformer network: A wearable-sensor-based fall detection method. *Sensors* **23**(14), 6360 (2023).
30. Agrawal, M. & Agrawal, S. Enhanced deep learning for detecting suspicious fall event in video data. *Intell. Automat. Soft Comput.* **36**, 3 (2023).
31. Chen, T., Li, X., Li, H. & Zhu, G. Deep learning-based fall detection using commodity Wi-Fi. *J. Inform. Intell.* **2**(4), 355–364 (2024).
32. Yhdego, H. G. Wearable sensor gait analysis for fall detection using deep learning methods. PhD Thesis, Old Dominion University, (2023).
33. Almukadi, W. S. et al. Deep feature fusion with computer vision driven fall detection approach for enhanced assisted living safety. *Sci. Rep.* **14**(1), 21537 (2024).
34. Sha, Y. et al. A novel lightweight deep learning fall detection system based on global-local attention and channel feature augmentation. *Interdisciplinary Nursing Res.* **2**(2), 68–75 (2023).
35. Cardenas, J. D., Gutierrez, C. A. & Aguilar-Ponce, R. Deep learning multi-class approach for human fall detection based on doppler signatures. *Int. J. Environ. Res. Public Health* **20**(2), 1123 (2023).
36. Min, X., Zhai, G., Gu, K., Liu, Y. & Yang, X. Blind image quality estimation via distortion aggravation. *IEEE Trans. Broadcast.* **64**(2), 508–517 (2018).
37. “Fall Detection Dataset > Overview,” Roboflow. Accessed: Feb. 26, 2025. [Online]. Available: <https://universe.roboflow.com/roboflow-universe-projects/fall-detection-ca3o8>
38. U. K. Kandagatla, “Fall detection dataset.” kaggle, 2022.
39. Adhikari, K., Bouchachia, H. & Nait-Charif, H. Activity recognition for indoor fall detection using convolutional neural network. In *2017 Fifteenth IAPR international conference on machine vision applications (MVA)*, 81–84 (IEEE, 2017).

Author contributions

S. Priya wrote the main manuscript text and K. Amshakala prepared Figs. 1, 2, 3, 4, 5, 6, 7, 8 and 9. All authors reviewed the manuscript.

Funding

Authors declare no funding for this research.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to S.P.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher’s note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025